Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

Implementation Solution Guide

Version 1.0.0 · 2026-07-17

Contents

Executive Summary — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) (Microsoft Azure) 4

What you get . . . 4

What this delivers . . . 4

The standard deployment model . . . 5

Outcome . . . 5

1. Solution Overview — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 5

What this solution is . . . 5

When to poll vs. when to use a webhook . . . 5

How it works . . . 6

Exactly-once: the two high-water-mark strategies . . . 6

Throttling and paging (keep a busy mailbox from failing runs) . . . 7

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 8

2. Requirements Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 8

2.1 An Azure Logic App (Consumption or Standard) . . . 8

2.2 A managed identity on the Logic App . . . 8

2.3 A least-privilege Graph application permission + admin consent . . . 9

2.4 The application permission scoped to ONE mailbox . . . 9

2.5 An Exchange Online target mailbox . . . 9

2.6 A durable store for the watermark . . . 9

2.7 Microsoft.Graph PowerShell (for the scripts) . . . 10

2.8 National-cloud endpoint decision . . . 10

Checklist . . . 10

Global Requirements — Azure Solutions 10

G1. A Microsoft 365 / Entra tenant . . . 11

G2. Administrative roles (least privilege) . . . 11

G3. Licensing baseline . . . 11

G4. Network connectivity to Microsoft 365 / Azure endpoints . . . 11

G5. Change control & maintenance window . . . 12

G6. Backup / rollback posture . . . 12

3. Implementation Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 12

Naming (set once, reuse verbatim) . . . 13
Step 1 — Create the Logic App and enable its managed identity . . . 13
Step 2 — Grant the least-privilege application permission and admin-consent it . . . 13
Step 3 — Scope the identity to the ONE target mailbox (concise; defer to product #4) . . . 14
Step 4 — Provision the durable watermark store . . . 14
Step 5 — Build the recurrence polling workflow . . . 14
Step 6 — Pilot on the one mailbox . . . 15
Step 7 — Expand . . . 15
5. Testing Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 16
Test 1 — Managed identity authenticates to Graph . . . 16
Test 2 — Least-privilege permission is scoped to the one mailbox . . . 16
Test 3 — Each message is processed exactly once across TWO runs . . . 16
Test 4 — The watermark advances and persists . . . 17
Test 5 — A burst does not double-process . . . 17
Test 6 — Throttling is handled gracefully (429) . . . 17
Test 7 — Idempotent re-run after interruption . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 18
A. Messages processed twice (double-processing) . . . 18
B. Messages skipped (never processed) . . . 18
C. Runs throttled / failing with HTTP 429 . . . 19
D. Watermark stuck or lost . . . 19
E. Graph read returns 403 (access denied) . . . 20
F. Managed identity can't authenticate to Graph . . . 20
G. Paging cut off / incomplete backlog . . . 20
H. Diagnostic scripts (quick reference) . . . 20
I. When to escalate . . . 21
7. Best Practices — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 21
8. Deliverables Summary — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) 22
Documentation . . . 22
PowerShell scripts (Microsoft.Graph) . . . 23
What the buyer receives . . . 23
Outcome . . . 23
Appendix A — PowerShell Scripts 24

Executive Summary — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account) (Microsoft Azure)

White-label · Azure Solutions

Not every environment can expose a public HTTPS endpoint for Microsoft Graph change notifications, and the legacy Office 365 Outlook connector cannot authenticate with a managed identity. When the event-driven webhook isn't an option, the robust fallback is a **scheduled Graph polling workflow** — but done **properly**, not a naive "get messages every 5 minutes" that double-processes mail and trips throttling. This solution builds a **recurrence-driven Azure Logic App** that reads new mail through Microsoft Graph with a **managed identity** (no service account, no stored secret), tracks a **high-water mark** so each message is processed **exactly once**, handles **throttling and paging**, and **marks or moves** processed items so a re-run is always safe.

This is a complete, **deploy-it-today** package, written for an administrator with no prior Logic Apps or Microsoft Graph experience.

What you get

- A plain-language **Solution Overview** of the recurrence Logic App, managed-identity authentication, the two exactly-once strategies (received-time watermark vs. `/messages/delta`), throttling/paging, and idempotent handling — plus when to poll vs. use the event-driven webhook (product #2).
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite (Logic App, managed identity, least-privilege application permission + admin consent, one-mailbox scoping, target mailbox, watermark store, Microsoft.Graph PowerShell, national-cloud endpoints).
- An **Implementation Guide** of precise, ordered steps — enable the managed identity, grant and consent the least-privilege permission, scope it to one mailbox (defer to product #4), provision the watermark store, and build the workflow with Graph read, 429 handling, `@odata.nextLink` paging, watermark persistence, and idempotent mark/move.
- Five **PowerShell scripts** (Microsoft.Graph) for discovery, PASS/FAIL validation, a processing/backlog report, an exactly-once watermark check, and a `-WhatIf`-gated app-role assignment helper.
- **Testing** and **Troubleshooting** guides for confident rollout.

What this delivers

A **recurrence-triggered Logic App** reading new messages via **Microsoft Graph** with a **managed identity**; a **high-water mark** (received-time watermark or delta `deltaLink`) that guarantees **exactly-once** processing; **throttle-safe** behavior that honors `Retry-After` and follows `@odata.nextLink` paging; and **idempotent** mark-as-read / move-to-`Processed` handling — with the application permission confined to **one mailbox** (product #4). Version-, throttling-, delta-, Consumption-vs-Standard-, and national-cloud-specific details are flagged to **verify against current Microsoft documentation** rather than asserted.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — with an **exactly-once validation gate** (two consecutive runs process each message once) and a documented rollback.

Outcome

A dependable, credential-free mailbox-polling workflow that reads each message exactly once over Graph — watermarked, throttle-safe, and idempotent — deployable by a single administrator and validated before you scale, and the right pattern precisely when an event-driven webhook cannot be used.

1. Solution Overview — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

What this solution is

This solution builds a **scheduled Microsoft Graph mailbox-polling workflow** in **Azure Logic Apps** that reads new mail from an Exchange Online mailbox on a timer, authenticating with the Logic App's **managed identity** and an **application permission** — no service account, no stored password, and no reliance on the legacy Office 365 Outlook connector. Crucially, it polls the mailbox **properly**: it tracks a **high-water mark** so every message is processed **exactly once**, it honors Graph **throttling and paging** so a busy mailbox never breaks a run, and it **marks or moves** processed items so a re-run is safe.

It is the **polling alternative** named inside the parent solution *"Exchange Online Mailbox Automation with Logic Apps Managed Identity — Replacing the O365 Connector with Microsoft Graph"* (**product #1** — the parent; do not re-derive its connector-replacement rationale here). It is deliberately the fallback to the event-driven design in *"Event-Driven Mail Processing with Microsoft Graph Change Notifications (Logic Apps + Managed Identity)"* (**product #2**). For confining the managed identity so it can read **only the one target mailbox**, this guide points to *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online"* (**product #4**) and does not reproduce its depth.

When to poll vs. when to use a webhook

- **Prefer the event-driven webhook (product #2)** when the environment can expose a **public HTTPS endpoint** for Graph **change notifications**. It is lower-latency (near-real-time) and does less work — Graph pushes only the changes.
- **Use this polling design (product #3)** when a public inbound endpoint is **not available or not permitted** (locked-down networks, no reverse proxy/public listener, policy against inbound webhooks), or when the workload only needs periodic batch processing. Polling makes **outbound** calls to Graph on a schedule, so it needs no inbound endpoint at all.
- Both designs share the **same identity spine**: a managed identity holding a **least-privilege application permission (e.g. `Mail.Read`)** scoped to **one mailbox** (product #4). Exact permission names and whether a lighter read scope suffices — **verify against current Microsoft documentation**.

> *The Office 365 Outlook connector **cannot authenticate with a managed identity** — whether any given Office 365 Outlook connector trigger/action supports managed identity is a moving target; **verify***

***against current Microsoft documentation**. This solution therefore calls **Microsoft Graph** directly via the Logic App **HTTP** action with **Managed identity** authentication.*

How it works

1. A **recurrence trigger** fires the Logic App on a schedule (e.g. every 5 minutes) — no inbound endpoint required.

2. The workflow **reads the stored watermark** (the high-water mark from the previous run) from a durable store.

3. It calls **Microsoft Graph** for **new messages only** using the watermark, authenticated by the Logic App's **managed identity** (the `Authorization` header is issued by Azure for the `https://graph.microsoft.com` audience — no secret in the workflow). Two supported high-water-mark strategies (see below).

4. It **follows paging** (`@odata.nextLink`) until all new messages for this run are retrieved, and **honors throttling** — on HTTP **429** it reads `Retry-After` and backs off rather than hammering Graph.

5. For each message it does the **business action** (the customer's own processing — create a ticket, save an attachment, notify a queue), then makes the item **idempotent**: **mark-as-read** and/or **move to a** `Processed` **folder** so it is never handled twice.

6. It **advances and persists the watermark** to the durable store so the next run starts exactly where this one ended — messages are neither skipped nor double-processed across runs.

Exactly-once: the two high-water-mark strategies

Choose one; the guide implements a watermark by default and documents delta as the alternative.

- **A) Watermark on** `receivedDateTime` **(default in this guide).** Persist the timestamp of the newest message processed. Each run queries `…/messages?$filter=receivedDateTime gt <watermark>&$orderby=receivedDateTime asc`, processes in order, then stores the newest `receivedDateTime`. **Tradeoffs:** simple, transparent, and store-agnostic; edge cases are **same-timestamp ties** (mitigate by also tracking the last processed message `id` and skipping it) and **clock/precision** granularity. Pair the watermark with **mark-as-read / move-to-**`Processed` so re-runs are idempotent even if the watermark write is interrupted.

- **B) Delta query (**`/messages/delta` **+** `deltaLink`**).** Graph returns a `@odata.deltaLink`; each run calls the stored `deltaLink` and Graph returns **only what changed** since last time, then hands back a fresh `deltaLink` to store. **Tradeoffs:** Graph tracks the change state for you (less filtering logic) but you must **persist and reuse the exact** `deltaLink`, handle its expiry/`410 Gone` (full re-sync), and understand that delta reflects **changes** (new, updated, deleted), not only arrivals. **Delta-query specifics — supported mailbox resources,** `deltaLink` **lifetime/expiry behavior, and paging semantics — must be verified against current Microsoft documentation** before relying on them; do not assume behavior.

Whichever strategy is chosen, the **mark/move step is the backstop**: because processed items are marked-as-read and/or moved out of the polled folder, a duplicated read (e.g. after a crash between "process" and "advance watermark") is caught and skipped — the workflow is idempotent by design.

Throttling and paging (keep a busy mailbox from failing runs)

• **Paging:** Graph returns messages in pages with an `@odata.nextLink`. The workflow **loops** on `nextLink` (an Until loop) until it is absent, so a backlog is fully drained in one run rather than silently truncated.

• **Throttling:** Graph enforces **per-app / per-mailbox request limits**; when exceeded it returns **HTTP 429** with a `Retry-After` header. The workflow **reads** `Retry-After` **and waits** that long before retrying, and uses **exponential backoff** on transient **5xx**. **Exact Graph throttling thresholds and limits change over time and must be verified against current Microsoft documentation** — never hard-code an assumed number; always defer to the returned `Retry-After`.

• **Page size:** use a modest `$top` (e.g. 25–50) so each page is cheap and a single run stays well within its execution window.

Benefits

• **No service account, no stored secret** — the managed identity is credentialless; nothing to rotate, leak, or have blocked by a password-expiry policy.

• **No public endpoint required** — pure outbound polling works in locked-down networks where a webhook (product #2) cannot.

• **Exactly-once processing** — the watermark plus idempotent mark/move means each message is handled once, even across restarts, throttling, and re-runs.

• **Throttle- and backlog-safe** — honoring `Retry-After` and following `@odata.nextLink` keeps a busy mailbox reliable.

• **Least-privilege** — one application permission (e.g. `Mail.Read`) scoped to **one mailbox** (product #4).

Limitations (be honest with buyers)

• **Polling has latency** — mail is picked up on the recurrence interval, not instantly. If near-real-time matters and a public endpoint is available, use the **event-driven webhook (product #2)**.

• **Application permissions are tenant-wide unless scoped** — `Mail.Read` as an application permission grants access to **all mailboxes** until you constrain it to the one target mailbox; that scoping is **mandatory** and is covered in depth by **product #4** (Application Access Policy and/or the newer Graph RBAC-for-Exchange model — which to use, and their exact behavior, **verify against current Microsoft documentation**).

• **Logic Apps Consumption vs. Standard differ** — trigger types, managed-identity support, private-networking, and the natural watermark store differ between **Consumption** (multi-tenant) and **Standard** (single-tenant, App Service). Which capabilities each offers **must be verified against current Microsoft documentation**; this guide notes where the choice matters.

• **National clouds differ** — Graph and login endpoints are different in **GCC High / DoD** and other national clouds (e.g. `graph.microsoft.us`). Endpoint hosts and feature parity **must be verified against current Microsoft documentation**.

• Polling reduces missed mail; it does not replace the mailbox's own retention, journaling, or compliance controls.

The standard deployment model

> ***Enable the Logic App's managed identity → grant the least-privilege application permission and admin-consent it → scope the identity to the one target mailbox (product #4) → build the recurrence Logic App (Graph read + paging + throttling + watermark + idempotent mark/move) → pilot on one mailbox → validate exactly-once → expand.*** *The repeatable pattern is* ***Assess → Configure → Pilot → Validate → Expand****, with an explicit* ***exactly-once validation gate*** *(two consecutive runs process each message once) and a documented rollback.*

2. Requirements Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

Verify **every** requirement before configuring. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-azure-solutions.md`. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout (see the naming block in `03-implementation.md`).

> ***Anything version-, feature-, or licensing-specific is flagged "verify against current Microsoft documentation" and is never asserted as fact*** *— exact Graph permission names/scopes, throttling limits, delta-query behavior, Consumption-vs-Standard capability differences, and national-cloud endpoints all change over time.*

2.1 An Azure Logic App (Consumption or Standard)

Description: An Azure Logic App to host the recurrence-driven workflow — named `Mail Polling – Support Mailbox – Pilot`. **Why:** The Logic App is the compute that runs the schedule, calls Graph, and holds the managed identity. **Verify:** `portal.azure.com` → **Logic apps**; run `GRAPHPOLL-Discovery.ps1`. **Implement-if-missing:** Create a Logic App. **Consumption (multi-tenant)** is the simplest for a single-mailbox poller; **Standard (single-tenant, App Service)** adds private networking and different trigger/state options. **The exact managed-identity, trigger, and networking differences between Consumption and Standard must be verified against current Microsoft documentation** — pick based on your networking and isolation needs, then keep the choice consistent through the guide.

2.2 A managed identity on the Logic App

Description: A **managed identity** for the Logic App — **system-assigned** (bound to this Logic App) or a **user-assigned** identity named `Mail Polling MI – Support Mailbox – Prod` reused across workflows. **Why:** The managed identity is what authenticates the HTTP-to-Graph calls; it replaces the service account and stores no secret. **Verify:** Logic App → **Identity** shows a system-assigned identity **On** (or the user-assigned identity attached); `GRAPHPOLL-Discovery.ps1` reports the identity and its object id. **Implement-if-missing:** Logic App → **Identity** → **System assigned** → **On** (or **User assigned** → **Add** the identity). Record the identity's **Object (principal) ID** — every later grant targets it.

2.3 A least-privilege Graph application permission + admin consent

Description: An **application** Graph permission granting mailbox read — e.g. `Mail.Read` — granted to the managed identity as an **app-role assignment**, with **admin consent**. **Why:** Polling from an unattended workflow uses **application** permissions (no signed-in user); the permission must be admin-consented and assigned to the identity, or every Graph call returns **403**. **Verify:** Entra → **Enterprise applications** → the managed identity → **Permissions**; or run `GRAPHPOLL-Validate.ps1` (confirms the app-role assignment exists). `GRAPHPOLL-Set-AppRole.ps1 -WhatIf` previews the assignment. **Implement-if-missing:** Assign the app role with `GRAPHPOLL-Set-AppRole.ps1` (defaults to `-WhatIf`; re-run with `-Execute`). **The exact least-privilege permission name and whether a narrower read scope than `Mail.Read` suffices must be verified against current Microsoft documentation** — do not assume; grant the narrowest that works.

2.4 The application permission scoped to ONE mailbox

Description: The managed identity's mailbox access **constrained to the single target mailbox** `supportmailbox@contoso.com`, via an **Application Access Policy** (mail-enabled security group `Mail Polling Scoped – Support Mailbox – Prod`) and/or the newer **Graph RBAC-for-Exchange** model. **Why:** An application `Mail.Read` grant reads **every** mailbox in the tenant until scoped. Constraining to one mailbox is the **security spine** of this solution. **Verify:** Test that the identity can read the target mailbox but is denied another; `GRAPHPOLL-Validate.ps1` checks the target is reachable. **Full scoping design/verification is covered by product #4** — not reproduced here. **Implement-if-missing:** Apply the scope per **product #4** *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online"*. **Which model to use (Application Access Policy vs. Graph RBAC-for-Exchange) and their exact behavior must be verified against current Microsoft documentation.**

2.5 An Exchange Online target mailbox

Description: The Exchange Online mailbox to poll — `supportmailbox@contoso.com` — with a `Processed` mail folder for idempotent moves. **Why:** This is the source of the messages the workflow reads and the destination folder that makes processing idempotent. **Verify:** `admin.microsoft.com` / Exchange admin center shows the mailbox; `GRAPHPOLL-Discovery.ps1` reports current message/backlog counts. **Implement-if-missing:** Confirm the licensed mailbox exists; create a child mail folder named `Processed` under the mailbox (the workflow moves handled items there).

2.6 A durable store for the watermark

Description: A place to persist the **high-water mark** (the `receivedDateTime` watermark or the delta `deltaLink`) between runs — options: a small **Azure Storage table/blob** (e.g. table `mailpollingwatermark`), a **Logic App Standard** stateful variable/workflow-state, or another durable key/value store. **Why:** The watermark **must survive between runs**; a lost or reset watermark causes skipped mail or reprocessing of the whole mailbox. **Verify:** The store is reachable and holds a readable watermark value; `GRAPHPOLL-Validate.ps1` checks the watermark store is reachable and `GRAPHPOLL-Report.ps1` reports the last-processed watermark. **Implement-if-missing:** Provision an **Azure Storage account** and a table named `mailpollingwatermark` (recommended for Consumption Logic Apps), granting the **same managed identity** the least-privilege data role to read/write it. **Which**

store is available/natural differs between Consumption and Standard and must be verified against current Microsoft documentation. Never store the watermark only in memory or a run-scoped variable.

2.7 Microsoft.Graph PowerShell (for the scripts)

Description: The **Microsoft.Graph** PowerShell SDK on the operator workstation for the discovery/validation/reporting/watermark/app-role scripts. **Why:** All five `GRAPHPOLL-*.ps1` scripts use Microsoft.Graph cmdlets to read grants, mailbox reachability, backlog, and to assign the app role. **Verify:** `Get-Module Microsoft.Graph -ListAvailable`; run `GRAPHPOLL-Discovery.ps1` (it imports the modules). **Implement-if-missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Sign in with an account holding the rights to read directory/app-role and (for `GRAPHPOLL-Set-AppRole.ps1`) to assign the app role and admin-consent.

2.8 National-cloud endpoint decision

Description: A decision on which **cloud** the tenant is in — commercial (`graph.microsoft.com`, `login.microsoftonline.com`) vs. a **national cloud** (e.g. GCC High / DoD: `graph.microsoft.us`, `login.microsoftonline.us`). **Why:** The Graph host, login authority, and Logic App HTTP audience differ by cloud; a workflow or script pointed at the wrong endpoint fails to authenticate or returns nothing. **Verify:** Confirm the tenant's cloud and its Graph/login endpoints. **The exact national-cloud endpoint hosts and feature parity must be verified against current Microsoft documentation.** **Implement-if-missing:** Set the HTTP action's Graph base URL, the managed-identity **audience**, and the Microsoft.Graph PowerShell **environment** (`Connect-MgGraph -Environment <…>`) to the correct cloud before building or running anything.

Checklist

- Azure Logic App `Mail Polling – Support Mailbox – Pilot` (Consumption vs Standard decided — verify) — `GRAPHPOLL-Discovery.ps1`
- Managed identity on the Logic App (system- or user-assigned) — object id recorded
- Least-privilege application Graph permission (e.g. `Mail.Read`) + admin consent — `GRAPHPOLL-Validate.ps1`
- Application permission **scoped to one mailbox** `supportmailbox@contoso.com` (per product #4)
- Exchange Online mailbox + a `Processed` folder
- Durable watermark store reachable (e.g. table `mailpollingwatermark`) — `GRAPHPOLL-Validate.ps1`
- Microsoft.Graph PowerShell installed
- National-cloud endpoints confirmed (verify)

Global Requirements — Azure Solutions

These requirements are common to **every** Azure Solutions solution in the standard program model. They are the baseline platform prerequisites that any Azure/Microsoft 365 engagement assumes. Each is

structured **Description / Why / Verify / Implement-if-missing**. Solution-specific requirements (see `02-requirements.md`) are layered on top of these.

> *Naming convention: every created object is named `<Purpose> – <Scope> – <Stage>` and reused verbatim throughout the engagement.*

G1. A Microsoft 365 / Entra tenant

• **Description:** An active Microsoft 365 / Microsoft Entra tenant for the cloud(s) in scope (commercial and/or government).

• **Why:** All identity, licensing, and service configuration is anchored to a tenant; nothing can be configured without one.

• **Verify:** Sign in to the relevant admin center (`https://admin.microsoft.com`) and confirm the tenant resolves and is active under **Settings** → **Org settings**.

• **Implement-if-missing:** Provision the tenant through the appropriate channel (commercial: standard; government: via an authorized AOS-G reseller).

G2. Administrative roles (least privilege)

• **Description:** Appropriately scoped admin roles for the work — Global Administrator for tenant-wide changes, and service-specific roles (Exchange, SharePoint, Teams, Azure Owner/Contributor) for day-to-day.

• **Why:** Every configuration and migration action requires a role with sufficient privilege; least-privilege scoping limits blast radius and satisfies audit requirements.

• **Verify:** In Entra → **Roles & admins**, confirm the operating accounts hold the required roles. In Azure → **Subscriptions** → **Access control (IAM)**, confirm RBAC assignments.

• **Implement-if-missing:** Assign the minimum roles needed; prefer eligible (PIM) activation over standing access where available.

G3. Licensing baseline

• **Description:** Sufficient Microsoft 365 and/or Azure licensing/credit for all in-scope users and workloads.

• **Why:** Features and capacity (mailboxes, SharePoint storage, AVD, VMs) are gated by license SKUs and subscription quota.

• **Verify: Billing** → **Licenses** for M365; **Subscriptions** → **Usage + quotas** for Azure. Compare assigned/available against the in-scope object count.

• **Implement-if-missing:** Procure the correct SKUs and request quota increases sized to the workload (add headroom for shared/resource objects).

G4. Network connectivity to Microsoft 365 / Azure endpoints

• **Description:** Reliable network egress to the required Microsoft 365 and Azure service endpoints (and the sovereign `.us` endpoints for government clouds), with sufficient bandwidth.

• **Why:** All cloud administration and any data movement depends on connectivity to the correct endpoint set; government clouds use distinct endpoints from commercial.

• **Verify:** Confirm reachability to the relevant endpoints (commercial `*.microsoft.com` / government `*.microsoft.us`, `*.azure.us`, `*.office365.us`). Measure sustained bandwidth against the data volume.

• **Implement-if-missing:** Open firewall/proxy allowances for the correct endpoint set; scale bandwidth or add ExpressRoute (matching the cloud — Gov ExpressRoute for Gov regions).

G5. Change control & maintenance window

• **Description:** An approved change-management process with scheduled maintenance windows and a stakeholder communications plan.

• **Why:** Cloud changes can be outward-facing and disruptive; controlled windows and comms reduce business impact and satisfy governance/compliance.

• **Verify:** Confirm an approved change record and a scheduled window for each disruptive action.

• **Implement-if-missing:** Stand up a lightweight change/approval process; schedule windows; prepare user comms templates.

G6. Backup / rollback posture

• **Description:** A defined backup and rollback position for any change — config export, data backup, and a documented back-out procedure before any disruptive action.

• **Why:** Cloud changes are not always reversible by default; a tested rollback is the safety net for cutovers and bulk operations.

• **Verify:** Confirm pre-change config/state is captured and a written rollback exists for each disruptive change.

• **Implement-if-missing:** Export current configuration/state; document and (where feasible) rehearse the rollback before executing the change.

3. Implementation Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

Follow these steps **in order**, in the **Azure portal** (`https://portal.azure.com`) and **Microsoft Entra admin center** (`https://entra.microsoft.com`). Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Enable the managed identity → grant + admin-consent the least-privilege application permission →* ***scope it to one mailbox (product #4)*** *→ build the recurrence Logic App (Graph read + paging + throttling + watermark + idempotent mark/move) → pilot → validate exactly-once → expand.*
> ***Verify-before-asserting:*** *Exact Graph permission names/scopes, throttling limits, delta-query behavior, Consumption-vs-Standard differences, and national-cloud endpoints all change — where this*

*guide flags **"verify against current Microsoft documentation,"** confirm before relying on it.*

Naming (set once, reuse verbatim)

- Logic App (poller): `Mail Polling – Support Mailbox – Pilot`
- Managed identity (if user-assigned): `Mail Polling MI – Support Mailbox – Prod`
- Target mailbox: `supportmailbox@contoso.com`
- Idempotency folder (child of the mailbox): `Processed`
- Mail-scoped security group (for one-mailbox scoping, product #4): `Mail Polling Scoped – Support Mailbox – Prod`
- Application permission granted to the identity: `Mail.Read` *(verify least-privilege name against current Microsoft documentation)*
- Watermark store (Azure Storage table): `mailpollingwatermark`

Step 1 — Create the Logic App and enable its managed identity

1. `portal.azure.com` → **Logic apps** → **Add**. Choose **Consumption** (simplest single-mailbox poller) or **Standard** (single-tenant, private networking) — **the trigger/identity/networking differences must be verified against current Microsoft documentation**; keep the choice consistent through this guide. Name it `Mail Polling – Support Mailbox – Pilot`. **Review + create.**

2. Open the Logic App → **Identity**.

- **System-assigned** (default in this guide): set **Status** → **On** → **Save**. Copy the **Object (principal) ID** shown.
- *Or* **User-assigned**: create/attach a user-assigned identity named `Mail Polling MI – Support Mailbox – Prod` and copy its **Object (principal) ID**.

1. Run `GRAPHPOLL-Discovery.ps1` to record the identity's object id, the target mailbox reachability, and the current backlog before any change.

Step 2 — Grant the least-privilege application permission and admin-consent it

1. Decide the permission — start with `Mail.Read` (application). **Whether a narrower read scope suffices must be verified against current Microsoft documentation.**

2. Preview the assignment: `GRAPHPOLL-Set-AppRole.ps1 -ManagedIdentityObjectId '<object id>' -PermissionName 'Mail.Read'` — it **defaults to** `-WhatIf` and prints exactly what it would grant.

3. Apply it: re-run with `-Execute`. This assigns the Graph app role to the managed identity **and** effects admin consent for that app-role assignment (an app-role assignment is itself the tenant admin's grant).

4. Confirm with `GRAPHPOLL-Validate.ps1` — the app-role assignment must be present. In the portal you can also check Entra → **Enterprise applications** → the managed identity → **Permissions**.

Step 3 — Scope the identity to the ONE target mailbox (concise; defer to product #4)

> *An application `Mail.Read` grant reads **every** mailbox until scoped. Scoping to the single mailbox is **mandatory** — the security spine of this solution. **Full design and verification are in product #4** *"Scoping Application-Permission Mailbox Access — Graph RBAC & Application Access Policies for Exchange Online"*; the concise version:*

1. Create the mail-enabled security group `Mail Polling Scoped – Support Mailbox – Prod` and add **only** `supportmailbox@contoso.com`.

2. Apply the scoping model — an **Application Access Policy** bound to that group, and/or the newer **Graph RBAC-for-Exchange** model. **Which model to use and its exact behavior must be verified against current Microsoft documentation (see product #4).**

3. Validate the scope: the identity can read `supportmailbox@contoso.com` and is **denied** any other mailbox. `GRAPHPOLL-Validate.ps1` confirms the target is reachable; product #4 covers the deny-other proof in depth.

Step 4 — Provision the durable watermark store

1. Create (or reuse) an **Azure Storage account** and add a **table** named `mailpollingwatermark`. *(For **Standard** Logic Apps a stateful workflow variable is an alternative — availability differs; **verify against current Microsoft documentation**.)*

2. Grant the **same managed identity** the least-privilege data role to read/write that table (so the workflow persists the watermark with the same credential-free identity).

3. Seed one row: `partitionKey=supportmailbox@contoso.com`, `rowKey=watermark`, `value=<a start time, e.g. now in ISO 8601 UTC>`. This is the initial high-water mark; the first run processes only mail newer than it.

Step 5 — Build the recurrence polling workflow

Open `Mail Polling – Support Mailbox – Pilot` → **Logic app designer** and build these actions in order:

1. **Recurrence** trigger — **Interval 5, Frequency Minute** (tune to the workload). No inbound endpoint is used.

2. **Read watermark** — action to read row `watermark` for `supportmailbox@contoso.com` from table `mailpollingwatermark`. Store it in a variable `Watermark`.

3. **HTTP — GET new messages (Graph, Managed identity):**

• **Method:** GET.

• **URI:** `https://graph.microsoft.com/v1.0/users/supportmailbox@contoso.com/mailFolders/Inbox/messages?$filter=receivedDateTime gt @{variables('Watermark')}&$orderby=receivedDateTime asc&$top=25`

(Use the national-cloud Graph host instead of `graph.microsoft.com` where applicable — verify.)

• **Authentication: Managed identity** → the Logic App's identity from Step 1 → **Audience** `https://graph.microsoft.com` *(national-cloud audience where applicable — verify)*. No secret is entered.

• **Delta alternative:** to use delta instead of the timestamp watermark, GET `…/mailFolders/Inbox/messages/delta`, store the returned `@odata.deltaLink`, and call that stored link next run. **Delta** `deltaLink` **lifetime/expiry (**`410 Gone` → **full re-sync) and supported resources must be verified against current Microsoft documentation.**

1. **Handle throttling (429):** add a condition on the HTTP action's status — on **429**, read the `Retry-After` response header, **Delay** for that many seconds, then retry. On transient **5xx**, retry with **exponential backoff**. Configure the action's built-in **retry policy** as a backstop. **Do not hard-code a throttling limit — always honor the returned** `Retry-After` **(exact limits vary; verify against current Microsoft documentation).**

2. **Page through results (Until loop):** set an **Until** loop that runs while the response body contains `@odata.nextLink` — each iteration does an **HTTP GET** on the `@odata.nextLink` (same Managed identity auth) and appends the returned `value` array, until `@odata.nextLink` is absent. This drains a backlog fully instead of truncating it.

3. **For each message** (`value` array), in order:

4. **Do the business action** — the customer's own processing (create a ticket, save an attachment, post to a queue). This is where the workflow does its real work.

5. **Make it idempotent** — **PATCH** `…/users/supportmailbox@contoso.com/messages/{id}` with `{ "isRead": true }` **and/or POST** `…/messages/{id}/move` with `{ "destinationId": "Processed" }` to move it to the `Processed` folder. Marking/moving means a re-read never double-processes.

6. **Advance the watermark** — after the loop, set the watermark to the **newest** `receivedDateTime` processed (for delta: store the new `@odata.deltaLink`). **Write** it back to row `watermark` in table `mailpollingwatermark`. Also record the last processed message `id` to break same-timestamp ties on the next run.

7. **Save** the workflow.

Step 6 — Pilot on the one mailbox

1. Send a few known test messages to `supportmailbox@contoso.com`.

2. **Run trigger** manually (or wait for the recurrence). Confirm each test message is processed and lands in `Processed` (or is marked read), and that the watermark row advanced.

3. Run `GRAPHPOLL-Report.ps1` to capture unread count, backlog, and the last-processed watermark.

4. Run `GRAPHPOLL-Test-Watermark.ps1` to inspect **two consecutive reads** and confirm **no overlap and no gap** (exactly-once). Complete the **Testing Guide** before expanding.

Step 7 — Expand

1. Once exactly-once is proven on `supportmailbox@contoso.com`, roll the pattern to additional mailboxes — **each mailbox gets its own scoped access (product #4) and its own watermark row**

(`partitionKey=<mailbox>`), so mailboxes never share a high-water mark.

2. Keep the same least-privilege identity model; re-validate each mailbox with `GRAPHPOLL-Validate.ps1` and track backlog with `GRAPHPOLL-Report.ps1`.

3. If a mailbox needs near-real-time processing and a public endpoint becomes available, consider moving it to the **event-driven webhook (product #2)** instead of polling.

Order of operations

`Create Logic App + enable MI → grant + consent least-priv app permission → scope to one mailbox (product #4) → provision watermark store → build recurrence workflow (read watermark → Graph GET → handle 429 → page nextLink → process + mark/move → advance watermark) → pilot → validate exactly-once → expand`

5. Testing Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

Run after building the workflow (Implementation Step 5) and piloting (Step 6), and again after each expansion. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names. The core proof is **exactly-once**: each message is processed **once** across two consecutive runs — never skipped, never doubled.

Test 1 — Managed identity authenticates to Graph

Do: Manually **Run trigger** on `Mail Polling – Support Mailbox – Pilot`; open the run history and inspect the HTTP-to-Graph action; or run `GRAPHPOLL-Validate.ps1`. **Expected:** The Graph GET returns **200** using **Managed identity** auth — no secret in the workflow, no service account. A **401/403** here means the app-role/consent (Step 2) or mailbox scope (Step 3) is wrong. **PASS/FAIL:** PASS if the read succeeds via managed identity with no stored credential.

Test 2 — Least-privilege permission is scoped to the one mailbox

Do: Confirm the identity reads `supportmailbox@contoso.com`; attempt (per product #4) to read a **different** mailbox. **Expected:** The target mailbox reads succeed; **any other mailbox is denied**. `GRAPHPOLL-Validate.ps1` confirms the target is reachable. **PASS/FAIL:** PASS only if the target is readable **and** a non-target mailbox is denied.

Test 3 — Each message is processed exactly once across TWO runs

Do: Send **three** known messages to `supportmailbox@contoso.com`. Run the workflow (**run A**). Without sending anything new, run it again (**run B**). Inspect which messages each run processed (run history and the `Processed` folder); or run `GRAPHPOLL-Test-Watermark.ps1`. **Expected: Run A** processes the three messages and moves/marks them; **run B** processes **zero** (they are past the watermark and out of the polled folder). No message is processed twice; none is skipped. **PASS/FAIL:** PASS only if total processings across A+B = 3 (each exactly once).

Test 4 — The watermark advances and persists

Do: Note the watermark row (`watermark` for `supportmailbox@contoso.com`) in table `mailpollingwatermark` before and after a run; or run `GRAPHPOLL-Report.ps1`. **Expected:** After a run that processed mail, the stored watermark equals the **newest processed** `receivedDateTime` (or the new `@odata.deltaLink`) and **survives** to the next run. **PASS/FAIL:** PASS if the watermark moved forward and is readable by the next run.

Test 5 — A burst does not double-process

Do: Send a **burst** of ~20–30 messages quickly, then run the workflow once (or let one recurrence fire). **Expected:** All burst messages are processed **once**; the `@odata.nextLink` paging loop drains the backlog in the run; the watermark advances to the newest. A second run processes none. **PASS/FAIL:** PASS if every burst message is processed exactly once and none is skipped by page truncation.

Test 6 — Throttling is handled gracefully (429)

Do: Drive enough volume to trigger throttling, or inspect a run that received **HTTP 429** from Graph in the run history. **Expected:** On 429 the workflow reads `Retry-After`, **waits**, and **retries** — the run **completes** rather than failing, and no message is lost or double-processed. **The exact throttling threshold is not assumed; the workflow defers to the returned** `Retry-After` **(verify limits against current Microsoft documentation). PASS/FAIL:** PASS if a throttled run recovers and still processes each message exactly once.

Test 7 — Idempotent re-run after interruption

Do: Simulate an interruption — e.g. process a message but interrupt before the watermark write; then run again. **Expected:** Because processed items are **marked read / moved to** `Processed`, the re-run does **not** re-process them even though the watermark may not have advanced; the mark/move backstop preserves exactly-once. **PASS/FAIL:** PASS if an interrupted-then-rerun sequence never double-processes.

Pass/fail gate

Gate	Requirement
Auth & scope pass	Tests 1–2 pass — managed identity reads via Graph and access is confined to the one mailbox
Exactly-once pass	Tests 3–5 pass — three messages processed once across two runs, watermark advances, a burst does not double-process
Resilience pass	Tests 6–7 pass — throttling recovers gracefully and an interrupted re-run stays idempotent

6. Troubleshooting Guide — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

Use `GRAPHPOLL-Discovery.ps1` / `GRAPHPOLL-Validate.ps1` for readiness state, `GRAPHPOLL-Report.ps1` for backlog + watermark, and `GRAPHPOLL-Test-Watermark.ps1` for an exactly-once proof. Objects are referenced by their `<Purpose> – <Scope> – <Stage>` names. Where a symptom depends on a Microsoft-versioned behavior (throttling limit, delta expiry, Consumption-vs-Standard, national-cloud endpoint), **verify against current Microsoft documentation** rather than assuming.

A. Messages processed twice (double-processing)

Symptom	Cause	Remediation
The same message is handled on consecutive runs	Watermark not advanced/persisted after processing	Confirm Step 5.7 writes the newest `receivedDateTime` back to row `watermark` in table `mailpollingwatermark`; run `GRAPHPOLL-Report.ps1` to see if the stored watermark moved
Duplicates after a crash mid-run	Processed but interrupted before watermark write	Rely on the **mark/move backstop** — ensure Step 5.6.2 marks-as-read and/or moves to `Processed`; a re-read of a moved/read item must be filtered out so it isn't re-handled
Same-timestamp messages re-read	Ties on `receivedDateTime` at the watermark boundary	Also store the last processed message `id` and skip it next run (Step 5.7); confirm with `GRAPHPOLL-Test-Watermark.ps1` (no overlap)

B. Messages skipped (never processed)

Symptom	Cause	Remediation
New mail is never picked up	Watermark set too far in the future, or never seeded	Reseed row `watermark` to a correct past time; `GRAPHPOLL-Report.ps1` shows watermark vs. newest message
A page of a backlog is missed	Paging loop not following `@odata.nextLink`	Confirm the **Until loop** (Step 5.5) iterates on `@odata.nextLink` until absent; `GRAPHPOLL-Test-Watermark.ps1` reports any GAP

Symptom	Cause	Remediation
Filter excludes valid mail	`$filter`/`$orderby` mismatch (e.g. wrong property or descending order)	Use `receivedDateTime gt <wm>` with `$orderby=receivedDateTime asc`; verify the property/query against current Microsoft documentation

C. Runs throttled / failing with HTTP 429

Symptom	Cause	Remediation
Run fails or errors under load	Graph 429 not handled	Add the 429 branch (Step 5.4): read `Retry-After`, **Delay**, then retry; enable the action retry policy. **Never hard-code a limit — defer to `Retry-After` (exact limits: verify against current Microsoft documentation)**
Repeated throttling every run	Interval too aggressive / page size too large	Lengthen the recurrence interval, lower `$top` (e.g. 25), and let paging drain the backlog across the run
5xx transient errors	Service blips	Exponential backoff retry on transient 5xx; the run should recover, not fail

D. Watermark stuck or lost

Symptom	Cause	Remediation
Watermark never changes	Write step failing (store permission or wrong row)	Confirm the managed identity has the least-priv data role on table `mailpollingwatermark`; `GRAPHPOLL-Validate.ps1` checks store reachability
Watermark reset to empty → whole mailbox reprocessed	Store cleared / new store without seed	Reseed the `watermark` row to the last-known-good time; the mark/move backstop limits damage (already-moved items aren't re-handled)
Standard workflow state lost	Relying on run-scoped variable instead of durable store	Persist the watermark in the **durable store** (table/blob/stateful workflow), never only in memory; verify Standard state options against current Microsoft documentation

E. Graph read returns 403 (access denied)

Symptom	Cause	Remediation
GET messages returns 403	App permission not granted/consented to the identity	Run `GRAPHPOLL-Set-AppRole.ps1 -Execute` to assign the app role (grants + consents); confirm with `GRAPHPOLL-Validate.ps1`
403 on the target mailbox specifically	The application permission is not **scoped** to include this mailbox	Apply the one-mailbox scope per **product #4** (Application Access Policy / Graph RBAC-for-Exchange); the group `Mail Polling Scoped – Support Mailbox – Prod` must include `supportmailbox@contoso.com`
Reads succeed on the WRONG mailbox too	Grant is unscoped (tenant-wide)	This is the security risk the solution prevents — scope to one mailbox immediately (product #4); `GRAPHPOLL-Validate.ps1` flags broader-than-needed grants

F. Managed identity can't authenticate to Graph

- Confirm the Logic App **Identity** is **On** (system-assigned) or the user-assigned identity `Mail Polling MI – Support Mailbox – Prod` is attached (Step 1).
- Confirm the HTTP action uses **Managed identity** auth with **Audience** `https://graph.microsoft.com` (or the correct **national-cloud** audience — verify).
- A **401** at the HTTP action points to identity/audience; a **403** points to permission/scope (section E).

G. Paging cut off / incomplete backlog

- If only the first page is processed, the **Until loop** (Step 5.5) isn't following `@odata.nextLink`. Ensure each iteration GETs the `@odata.nextLink` with the same Managed identity auth and stops only when it's absent.
- `GRAPHPOLL-Test-Watermark.ps1` will report a GAP if messages newer than the watermark are missed by the paging loop.

H. Diagnostic scripts (quick reference)

```
# Readiness: identity grants, mailbox reachability, backlog
powershell -File .\GRAPHPOLL-Discovery.ps1 -ManagedIdentityObjectId '<obj id>' -Mailbox 'supportmailbox@co

# PASS/FAIL gates: identity + least-priv permission + mailbox in scope + watermark store reachable
powershell -File .\GRAPHPOLL-Validate.ps1 -ManagedIdentityObjectId '<obj id>' `
    -WatermarkTableUri 'https://<account>.table.core.windows.net/mailpollingwatermark'

# Backlog + last-processed watermark (+ CSV)
```

```
powershell -File .\GRAPHPOLL-Report.ps1 -Mailbox 'supportmailbox@contoso.com' -StoredWatermark '<iso8601 u

# Exactly-once: two consecutive reads, no overlap / no gap
powershell -File .\GRAPHPOLL-Test-Watermark.ps1 -Mailbox 'supportmailbox@contoso.com' -StartWatermark '<is

# Assign the least-priv application permission (DEFAULTS TO -WhatIf; add -Execute to apply)
powershell -File .\GRAPHPOLL-Set-AppRole.ps1 -ManagedIdentityObjectId '<obj id>' -PermissionName 'Mail.Rea
```

I. When to escalate

If the managed identity is On, the app role is granted and scoped to the one mailbox, the watermark store is reachable and seeded, throttling is handled via `Retry-After`, and mail is still skipped or double-processed after a clean `GRAPHPOLL-Test-Watermark.ps1` run, collect `GRAPHPOLL-Report.ps1` output, the Logic App run history for the affected run, and the workflow definition, and open a Microsoft support case. Re-confirm any throttling/delta/national-cloud assumption against current Microsoft documentation before escalating.

7. Best Practices — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

• **Prefer the webhook when you can; poll when you can't.** If a public HTTPS endpoint is available and near-real-time matters, use the event-driven design (**product #2**). Reach for polling when there is no inbound endpoint or the workload is periodic — polling is all **outbound** and needs no public listener.

• **Never poll without a watermark.** A naive "get messages every 5 minutes" double-processes mail and trips throttling. Always persist a **high-water mark** (`receivedDateTime` watermark or delta `deltaLink`) so each run reads only what's new.

• **Make the mark/move step the exactly-once backstop.** The watermark alone can miss an edge case (crash between "process" and "advance watermark"); **mark-as-read and/or move to** `Processed` so a re-read is caught and skipped. Watermark **and** mark/move together = idempotent.

• **Honor** `Retry-After`**, never a hard-coded limit.** On HTTP 429, read `Retry-After` and wait; back off exponentially on transient 5xx. **Graph throttling limits change — verify against current Microsoft documentation and always defer to the returned header.**

• **Always follow** `@odata.nextLink`**.** Loop on paging until the link is absent so a backlog is fully drained; a single un-paged GET silently truncates a busy mailbox.

• **Least-privilege, one mailbox — the spine.** Grant the narrowest application read permission (e.g. `Mail.Read`; verify a narrower scope) and **scope it to the single mailbox** via Application Access Policy or Graph RBAC-for-Exchange (**product #4**). An unscoped application grant reads the whole tenant.

• **Credential-free by design.** Authenticate the HTTP-to-Graph calls with the Logic App's **managed identity** — no service account, no secret in the workflow, nothing to rotate or leak.

• **One watermark row per mailbox.** When you expand, give each mailbox its own watermark (`partitionKey=<mailbox>`) so mailboxes never share a high-water mark and can't skip each other's mail.

• **Name everything and reuse the name.** Use `<Purpose> – <Scope> – <Stage>` for the Logic App, identity, scoped group, and `Processed` folder, and reuse it verbatim across scripts, tests, and

rollback so the estate stays auditable.

• **Confirm the cloud first.** Point the HTTP action, managed-identity audience, and Microsoft.Graph PowerShell at the correct **national-cloud** endpoints before building — commercial and GCC High / DoD differ; **verify against current Microsoft documentation**.

• **Keep** `$top` **modest.** A small page size (e.g. 25–50) keeps each page cheap and the run inside its execution window; let the paging loop, not a large single page, handle volume.

• **Pilot, then expand.** Prove exactly-once on one mailbox (two consecutive runs) before rolling the pattern out — the same repeatable shape every time.

8. Deliverables Summary — Graph Mailbox Polling Workflows in Azure Logic Apps (Managed Identity, No Service Account)

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	What Graph mailbox polling is — recurrence Logic App, managed identity, exactly-once watermark/delta, throttling + paging, idempotent mark/move; when to poll vs webhook (product #2); pointers to #1 (parent) and #4 (scoping); benefits, limitations
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing (Logic App Consumption/Standard, managed identity, least-priv app permission + consent, one-mailbox scope, target mailbox, watermark store, Microsoft.Graph PowerShell, national-cloud)
Global Requirements (Azure Solutions)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps — enable MI, grant + consent app permission, scope to one mailbox (defer #4), provision watermark store, build the recurrence workflow (Graph read + 429 handling + nextLink paging + watermark + mark/move), pilot
5. Testing Guide	Exactly-once validation with PASS/FAIL gates — each message once across two runs, watermark advances, burst not doubled, throttling handled
6. Troubleshooting Guide	Field issues and remediation (double-processing, skipped mail, 429 runs, watermark stuck/lost, 403 read/scope, paging cut off)
7. Best Practices	Field-proven recommendations

PowerShell scripts (Microsoft.Graph)

Script	What it does
`GRAPHPOLL-Discovery.ps1`	Read-only: the managed identity's app-permission grants, target mailbox reachability, and current message/backlog counts
`GRAPHPOLL-Validate.ps1`	PASS/FAIL: managed identity + least-privilege permission present, target mailbox in scope, workflow present, watermark store reachable
`GRAPHPOLL-Report.ps1`	Processing/backlog report — unread count and last-processed watermark — plus a CSV export
`GRAPHPOLL-Test-Watermark.ps1`	Read-only: inspects two consecutive reads to confirm exactly-once (no overlap, no gap)
`GRAPHPOLL-Set-AppRole.ps1`	The only state-changer: assigns/verifies the application permission (app-role) on the managed identity; **defaults to** `-WhatIf`

What the buyer receives

- This implementation guide as a **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable process following the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with an **exactly-once validation gate** and a documented rollback.

Outcome

A dependable, credential-free mailbox-polling workflow that reads each message **exactly once** over Microsoft Graph — watermarked, throttle-safe, and idempotent — the right pattern when an event-driven webhook (product #2) isn't available, with the managed identity confined to one mailbox (product #4).

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GRAPHPOLL-Discovery.ps1

Purpose: Read-only discovery. For the Logic App's managed identity (by object id) it lists the Microsoft Graph application (app-role) permissions granted, confirms the target mailbox supportmailbox@contoso.com is reachable, and reports current total / unread message counts so you can see the backlog before building the poller. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Discovery.ps1 `-ManagedIdentityObjectId '<managed identity object id>'` -Mailbox 'supportmailbox@contoso.com' Requires the Microsoft.Graph PowerShell SDK. The script runs Connect-MgGraph if not already connected. For a national cloud (e.g. GCC High / DoD), pass -Environment (e.g. 'USGov', 'USGovDoD') so it targets the correct Graph endpoint — verify the exact name against current Microsoft documentation. GRAPH SCOPES (delegated, for this script's read-only sign-in) Application.Read.All, Directory.Read.All, Mail.Read (or Mail.ReadBasic.All) — read-only. (The workflow itself uses an APPLICATION Mail.Read permission on the managed identity; this script only reads configuration and counts.) EXPECTED RESULTS A snapshot: the managed identity's granted Graph app roles, whether the target mailbox resolves and is readable, and total/unread message counts — the readiness picture before you build the recurrence polling workflow. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers and counts only; makes no changes to identities, permissions, mailboxes, or messages. White-label: examples use contoso.com only. Version-, scope-, throttling-, and national-cloud-specific details: verify against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, [string]$Mailbox = 'supportmailbox@contoso.com', [string]$Environment = 'Global' # e.g. 'USGov' / 'USGovDoD' for national clouds — verify exact name)

How to run: powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Discovery.ps1 `-ManagedIdentityObjectId '<managed identity object id>'` -Mailbox 'supportmailbox@contoso.com' Requires the Microsoft.Graph PowerShell SDK. The script runs Connect-MgGraph if not already connected. For a national cloud (e.g. GCC High / DoD), pass -Environment (e.g. 'USGov', 'USGovDoD') so it targets the correct Graph endpoint — verify the exact name against current Microsoft documentation. GRAPH SCOPES (delegated, for this script's read-only sign-in) Application.Read.All, Directory.Read.All, Mail.Read (or Mail.ReadBasic.All) — read-only. (The workflow itself uses an APPLICATION Mail.Read permission on the managed identity; this script only reads configuration and counts.) EXPECTED RESULTS A snapshot: the managed identity's granted Graph app roles, whether the target mailbox resolves and is readable, and total/unread message counts — the readiness picture before you build the recurrence polling workflow. .NOTES Azure Solutions — read-only. The repeatable way. Reports on configuration/identifiers and counts only; makes no changes to identities, permissions, mailboxes, or messages. White-label: examples use contoso.com only. Version-, scope-, throttling-, and national-cloud-specific details: verify against current Microsoft documentation. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, [string]$Mailbox = 'supportmailbox@contoso.com', [string]$Environment = 'Global' # e.g. 'USGov' / 'USGovDoD' for national clouds — verify exact name)

GRAPHPOLL-Report.ps1

Purpose: Read-only reporting. Produces a per-mailbox snapshot: Inbox total / unread count, the count in the 'Processed' idempotency folder, the newest message receivedDateTime, and the last-processed watermark currently stored — so you can see backlog trend and confirm the high-water mark is advancing. Exports the same to CSV. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Report.ps1 `-Mailbox 'supportmailbox@contoso.com'` -StoredWatermark '2026-07-17T12:00:00Z' ` -CsvPath .\GRAPHPOLL-report.csv Requires the Microsoft.Graph PowerShell SDK (Connect-MgGraph runs if not connected). Pass the current -StoredWatermark you read from your watermark store (table 'mailpollingwatermark'), or omit it to report counts only. For a national cloud, pass -Environment. GRAPH SCOPES (delegated, read-only sign-in) Mail.Read (or Mail.ReadBasic.All), Directory.Read.All. EXPECTED RESULTS A console table + CSV: Inbox total/unread, Processed count, newest receivedDateTime, and the stored watermark — the evidence that the poller is keeping up and the watermark is moving forward. .NOTES Azure Solutions — read-only reporting. The repeatable way. Reports only; makes no changes to mailboxes, messages, or the watermark. White-label: contoso.com only. Backlog thresholds are informational; verify Graph limits against current Microsoft documentation. > [CmdletBinding()] param([string]$Mailbox = 'supportmailbox@contoso.com', [string]$StoredWatermark, [string]$CsvPath = ".\GRAPHPOLL-report.csv", [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Report.ps1 `-Mailbox 'supportmailbox@contoso.com'` -StoredWatermark '2026-07-17T12:00:00Z' ` -CsvPath .\GRAPHPOLL-report.csv Requires the Microsoft.Graph PowerShell SDK (Connect-MgGraph runs if not connected). Pass the current -StoredWatermark you read from your watermark store (table 'mailpollingwatermark'), or omit it to report counts only. For a national cloud, pass -Environment. GRAPH SCOPES (delegated, read-only sign-in) Mail.Read (or Mail.ReadBasic.All), Directory.Read.All. EXPECTED RESULTS A console table + CSV: Inbox total/unread, Processed count, newest receivedDateTime, and the stored watermark — the evidence that the poller is keeping up and the watermark is moving forward. .NOTES Azure Solutions — read-only reporting. The repeatable way. Reports only; makes no changes to mailboxes, messages, or the watermark. White-label: contoso.com only. Backlog thresholds are informational; verify Graph limits against current Microsoft documentation. > [CmdletBinding()] param([string]$Mailbox = 'supportmailbox@contoso.com', [string]$StoredWatermark, [string]$CsvPath = ".\GRAPHPOLL-report.csv", [string]$Environment = 'Global')

GRAPHPOLL-Set-AppRole.ps1

Purpose: Grants the managed identity the application permission the polling workflow needs, as an app-role assignment on the Microsoft Graph service principal. An app-role assignment IS the tenant admin's consent for that application permission, so this both grants and consents in one step. DEFAULTS TO -WhatIf: it prints exactly what it would assign and changes nothing until you pass -Execute. This grant is tenant-wide for the permission until you SCOPE it to one mailbox — do that per product #4 "Scoping Application-Permission Mailbox Access". Granting Mail.Read here without scoping lets the identity read every mailbox; scope it before you pilot. HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Set-AppRole.ps1 `-ManagedIdentityObjectId '<managed identity object id>'` -PermissionName 'Mail.Read' Apply for real (explicit): ... -Execute Requires the Microsoft.Graph PowerShell SDK. The signed-in account needs rights to assign app roles / grant application permissions (e.g. Privileged Role Administrator or Global Administrator). For a national cloud, pass -Environment. GRAPH SCOPES (delegated, for THIS script's sign-in) AppRoleAssignment.ReadWrite.All, Application.Read.All, Directory.Read.All. (These let the script create the app-role assignment; the workflow itself then uses the APPLICATION Mail.Read permission this script grants.) EXPECTED RESULTS In -WhatIf mode (default): a printed description of the app-role assignment

it would create (managed identity -> Graph app role <PermissionName>) — nothing changes. With -Execute: the assignment is created (idempotent — if it already exists it reports so). Verify the exact least-privilege permission name against current Microsoft documentation before granting. .NOTES Azure Solutions — DEFAULTS TO -WhatIf. The repeatable way. The only state-changing script in this solution. Grants exactly one named app role to one identity; touches nothing else. Scope the grant to ONE mailbox afterward (product #4). White-label: contoso.com only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, [string]$PermissionName = 'Mail.Read', # verify least-privilege name against current MS docs [string]$Environment = 'Global', [switch]$Execute # omit for -WhatIf (default safe mode))

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Set-AppRole.ps1 `-ManagedIdentityObjectId '<managed identity object id>'` -PermissionName 'Mail.Read' Apply for real (explicit): ... -Execute Requires the Microsoft.Graph PowerShell SDK. The signed-in account needs rights to assign app roles / grant application permissions (e.g. Privileged Role Administrator or Global Administrator). For a national cloud, pass -Environment. GRAPH SCOPES (delegated, for THIS script's sign-in) AppRoleAssignment.ReadWrite.All, Application.Read.All, Directory.Read.All. (These let the script create the app-role assignment; the workflow itself then uses the APPLICATION Mail.Read permission this script grants.) EXPECTED RESULTS In -WhatIf mode (default): a printed description of the app-role assignment it would create (managed identity -> Graph app role <PermissionName>) — nothing changes. With -Execute: the assignment is created (idempotent — if it already exists it reports so). Verify the exact least-privilege permission name against current Microsoft documentation before granting. .NOTES Azure Solutions — DEFAULTS TO -WhatIf. The repeatable way. The only state-changing script in this solution. Grants exactly one named app role to one identity; touches nothing else. Scope the grant to ONE mailbox afterward (product #4). White-label: contoso.com only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, [string]$PermissionName = 'Mail.Read', # verify least-privilege name against current MS docs [string]$Environment = 'Global', [switch]$Execute # omit for -WhatIf (default safe mode))

GRAPHPOLL-Test-Watermark.ps1

Purpose: Read-only verification of the high-water-mark logic for the "Graph Mailbox Polling Workflows" solution. It performs the SAME query the Logic App uses — messages with receivedDateTime gt <watermark>, ordered ascending — for two consecutive "runs": Run A: reads with the supplied -StartWatermark, then computes the watermark it WOULD store (the newest receivedDateTime it saw). Run B: reads again with that computed watermark. It then asserts EXACTLY-ONCE: - No OVERLAP: no message id appears in both Run A and Run B. - No GAP: every message strictly newer than the Run-A watermark is picked up by Run B (nothing silently skipped between the two reads). It reads only message metadata (id, receivedDateTime, subject); it does NOT process, mark, move, or write any watermark. Nothing is changed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Test-Watermark.ps1 `-Mailbox 'supportmailbox@contoso.com'` -StartWatermark '2026-07-17T00:00:00Z' ` -PageSize 25 Requires the Microsoft.Graph PowerShell SDK (Connect-MgGraph runs if not connected). For a national cloud, pass -Environment. Send a couple of test messages between observing Run A and Run B if you want to prove the "no gap" arm with live arrivals. GRAPH SCOPES (delegated, read-only sign-in) Mail.Read (or Mail.ReadBasic.All). EXPECTED RESULTS A PASS/FAIL: PASS when the two reads have no overlapping message ids AND no message newer than the Run-A watermark is missed by Run B — i.e. each message is accounted for exactly once. FAIL prints the offending ids so you can see a double-read (overlap) or a skip (gap). .NOTES Azure Solutions — read-only exactly-once check. The repeatable way. Simulates/inspects reads only; never

processes, marks, moves, or persists a watermark. White-label: contoso.com only. Delta-query behavior differs from timestamp watermarking — verify against current Microsoft documentation if you adopt /messages/delta. > [CmdletBinding()] param([string]$Mailbox = 'supportmailbox@contoso.com', [Parameter(Mandatory)] [string]$StartWatermark, [int]$PageSize = 25, [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Test-Watermark.ps1 `-Mailbox 'supportmailbox@contoso.com'` -StartWatermark '2026-07-17T00:00:00Z' ` -PageSize 25 Requires the Microsoft.Graph PowerShell SDK (Connect-MgGraph runs if not connected). For a national cloud, pass -Environment. Send a couple of test messages between observing Run A and Run B if you want to prove the "no gap" arm with live arrivals. GRAPH SCOPES (delegated, read-only sign-in) Mail.Read (or Mail.ReadBasic.All). EXPECTED RESULTS A PASS/FAIL: PASS when the two reads have no overlapping message ids AND no message newer than the Run-A watermark is missed by Run B — i.e. each message is accounted for exactly once. FAIL prints the offending ids so you can see a double-read (overlap) or a skip (gap). .NOTES Azure Solutions — read-only exactly-once check. The repeatable way. Simulates/inspects reads only; never processes, marks, moves, or persists a watermark. White-label: contoso.com only. Delta-query behavior differs from timestamp watermarking — verify against current Microsoft documentation if you adopt /messages/delta. > [CmdletBinding()] param([string]$Mailbox = 'supportmailbox@contoso.com', [Parameter(Mandatory)] [string]$StartWatermark, [int]$PageSize = 25, [string]$Environment = 'Global')

GRAPHPOLL-Validate.ps1

Purpose: Read-only validation. Confirms, as PASS/FAIL gates: 1) the Logic App's managed identity exists, 2) it holds a least-privilege Graph application permission (e.g. Mail.Read) — and flags any broader-than-needed grant, 3) the target mailbox supportmailbox@contoso.com is reachable (i.e. the scope lets the identity read it), 4) the watermark store (Azure Storage table 'mailpollingwatermark') is reachable and holds a readable watermark for the mailbox. Makes no changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Validate.ps1 `-ManagedIdentityObjectId '<managed identity object id>'` -Mailbox 'supportmailbox@contoso.com' `-ExpectedPermission 'Mail.Read'` -WatermarkTableUri 'https://<account>.table.core.windows.net/mailpollingwatermark' Requires the Microsoft.Graph PowerShell SDK (Connect-MgGraph runs if not connected). The watermark-store check is best-effort against the table endpoint; adapt -WatermarkTableUri to your store (table/blob/other). For a national cloud, pass -Environment. GRAPH SCOPES (delegated, read-only sign-in) Application.Read.All, Directory.Read.All, Mail.Read (or Mail.ReadBasic.All). EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Non-zero exit if any gate FAILS — so it can be used in a readiness pipeline. Verify exact permission names / scope model against current Microsoft documentation. .NOTES Azure Solutions — read-only PASS/FAIL. The repeatable way. Reports only; makes no changes. Least-privilege application permission scoped to ONE mailbox is the security spine — a broader grant is flagged. White-label: contoso.com only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, [string]$Mailbox = 'supportmailbox@contoso.com', [string]$ExpectedPermission = 'Mail.Read', [string]$WatermarkTableUri, [string]$Environment = 'Global')

How to run: powershell -ExecutionPolicy Bypass -File .\GRAPHPOLL-Validate.ps1 `-ManagedIdentityObjectId '<managed identity object id>'` -Mailbox 'supportmailbox@contoso.com' `-ExpectedPermission 'Mail.Read'` -WatermarkTableUri 'https://<account>.table.core.windows.net/mailpollingwatermark' Requires the Microsoft.Graph PowerShell SDK (Connect-MgGraph runs if not connected). The watermark-store check is best-effort against the table endpoint; adapt -WatermarkTableUri to your store (table/blob/other). For a national cloud, pass

-Environment. GRAPH SCOPES (delegated, read-only sign-in) Application.Read.All, Directory.Read.All, Mail.Read (or Mail.ReadBasic.All). EXPECTED RESULTS A PASS/FAIL line per gate and an overall result. Non-zero exit if any gate FAILS — so it can be used in a readiness pipeline. Verify exact permission names / scope model against current Microsoft documentation. .NOTES Azure Solutions — read-only PASS/FAIL. The repeatable way. Reports only; makes no changes. Least-privilege application permission scoped to ONE mailbox is the security spine — a broader grant is flagged. White-label: contoso.com only. > [CmdletBinding()] param([Parameter(Mandatory)] [string]$ManagedIdentityObjectId, [string]$Mailbox = 'supportmailbox@contoso.com', [string]$ExpectedPermission = 'Mail.Read', [string]$WatermarkTableUri, [string]$Environment = 'Global')